Functions and Variables

1. Hello World

```
Hello, World!
```

Anatomy of `print()`

```
print("Hello, World!")
```

- **Function**: a named block of code that does a task
  - `print()` : prints the argument to the screen
- **Argument**: input to the function
  - `"Hello, World!"`
- **Side effect**: what the function does to the outside world
  - print to the screen

Bugs 🐞

```
print("Hello, World!"
```

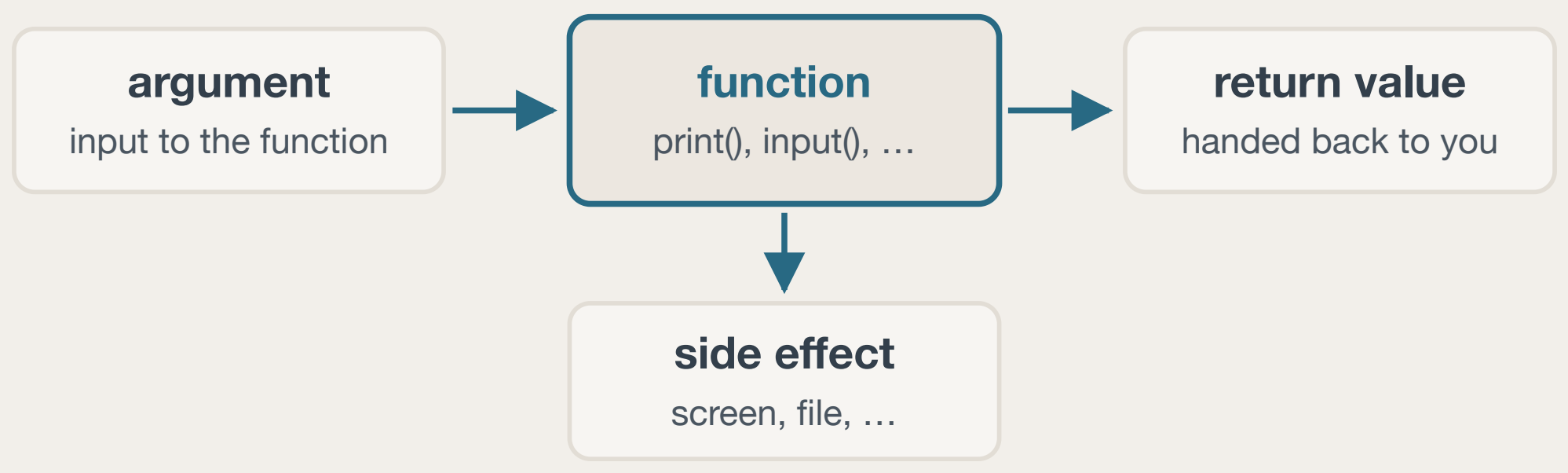
```
  Cell In[1], line 1
    print("Hello, World!"
                          ^
SyntaxError: incomplete input
```

2. Hello to You

```
What's your name? John
Hello, John
```

Anatomy of `input()`

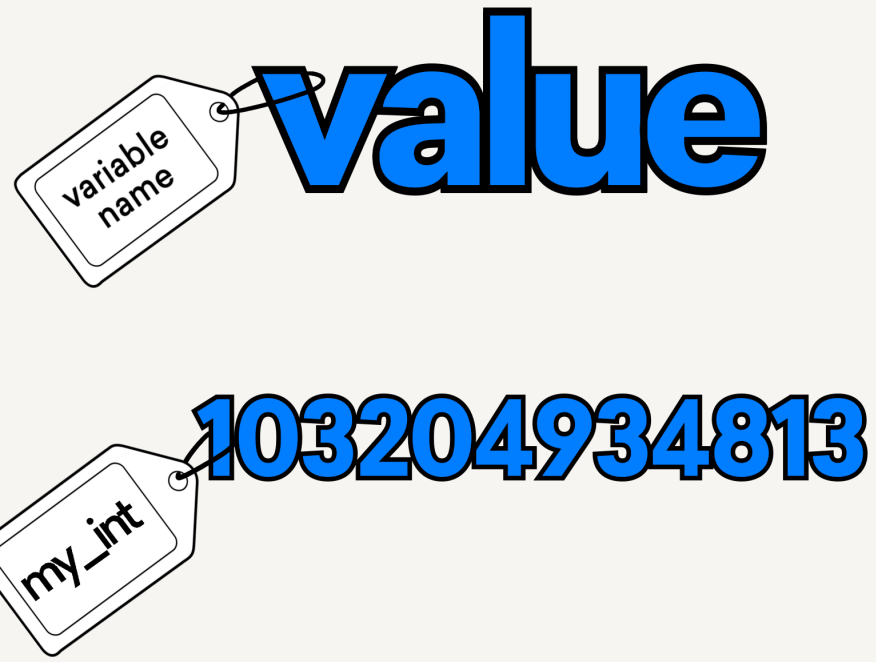
```
answer = input("What's your name? ")
# function:       input() to get user input
# argument:       "What's your name? "
# side effect:    shows the prompt and waits for typing
# return value:   what the user typed, named as answer
```



Variables: name tags for values

```
name = "John"   # the tag name points to a value
name = "Emily"  # the tag moves to a new value
print(name)     # Emily
```

`=` **assigns**: work out the right side, then attach the name on the left



🤔 Printing a variable

```
answer = input("What's your name? ")
print("Hello, answer")
```

Joining strings and variables (`+`)

```
answer = input("What's your name? ")

greeting = "Hello, " + answer   # space inside the string
print(greeting)
```

Joining strings and variables (f-string)

```
answer = input("What's your name? ")

greeting = f"Hello, {answer}"    # f before the quote; {} is filled in
print(greeting)                  # Hello, John

print("Hello, {answer}")         # forgot the f: prints Hello, {answer}
```

Printing multiple arguments

```
answer = input("What's your name? ")

# no space inside the string
print("Hello,", answer)
```

`help()` to look up function details

```
help(print)
```

```
Help on built-in function print in module builtins:

print(*args, sep=' ', end='\n', file=None, flush=False)
    Prints the values to a stream, or to sys.stdout by default.

    sep
      string inserted between values, default a space.
    end
      string appended after the last value, default a newline.
    file
      a file-like object (stream); defaults to the current sys.stdout.
    flush
      whether to forcibly flush the stream.
```

Or, you can refer to the documentation online:

https://docs.python.org/3/library/functions.html#print

Comments (`#`)

```
# + operator
print("Hello, " + answer)

# f-string
print(f"Hello, {answer}")

# multiple arguments
print("Hello,", answer)
```

💻 3. Self-introduction

Requirements:

- Use `input()` to ask for the user's name and age, and store each in its own variable.
- Use `print()` to display `Hello, my name is [name]. I am [age] years old.`

Expected Outputs:

```
What's your name? Emily
What's your age? 25
Hello, my name is Emily. I am 25 years old.
```

4. Messy input

```
What's your name?     john
Hello, John

What's your name? jAnE doE
Hello, Jane Doe
```

String methods

A method is a function that belongs to a value.

```
answer = input("What's your name? ")  # answer refers to a string

print(answer)      # function: print answer

answer.strip()     # method: answer's strip function (read the dot as 's)
```

https://docs.python.org/3/library/stdtypes.html#string-methods

`strip()` to remove surrounding spaces

```
answer = input("What's your name? ")   # user types "   john  "

print(answer.strip())          # "john": strip() returns a cleaned copy
print(answer)                 # "   john  ": answer is unchanged

answer = answer.strip()       # point answer at the cleaned copy
print("Hello, " + answer)     # Hello, john
```

`capitalize()` to uppercase the first letter, lowercase the rest

```
answer = input("What's your name? ")   # user types "   jAnE doE  "

answer = answer.strip().capitalize()   # "Jane doe"
# chaining: strip() returns a string, so capitalize() can be called on it

print("Hello, " + answer)              # Hello, Jane doe
```

`title()` to capitalize each word

```
answer = input("What's your name? ")    # user types "   jAnE doE  "
answer = answer.title()                 # "   Jane Doe  ": title() doesn't strip
print("Hello, " + answer)               # Hello,    Jane Doe
```

5. Hello Function

```
hello()
# Output: Hello, world

hello("John")
# Output: Hello, John
```

`def` to define a function

```
def hello():                  # def, a name, (), and a colon
    print("Hello, world")     # body: indented (a tab), so it belongs to hello()
print("Bye")                  # not indented: not part of hello(), runs right away
hello()                       # call: now the body runs
```

Defining stores the code. Calling runs it.

Parameters to accept arguments

- **Parameter**: a placeholder in the definition (`name`)
- **Argument**: the value passed in the call (`"John"`)

```
def hello(name):            # name: parameter
    print("Hello,", name)

# Call the function with an argument
hello("John")               # name = "John"
hello("Emily")              # name = "Emily"
```

Positional vs. keyword arguments

```
def greet(greeting, name):
    print(greeting + ",", name)

# positional: matched by order
# 1st argument → 1st parameter, 2nd → 2nd
greet("Hello", "John")      # greeting = "Hello", name = "John" → Hello, John
greet("John", "Hello")      # greeting = "John", name = "Hello" → John, Hello

# keyword: matched by parameter name, so order doesn't matter
# 1st argument → greeting, 2nd → name
greet(name="John", greeting="Hello")    # → Hello, John
```

Parameters with default values

```
def hello(name):            # name: parameter
    print("Hello,", name)

hello()                     # TypeError: hello() missing argument: 'name'
```

```
def hello(name="world"):  # name: parameter with default value "world"
    print("Hello,", name)

# Call the function with an argument
hello("John")   # name = "John"

# Call the function without an argument
hello()         # name = "world"
```

🤔 What does each call print?

```
def hello(a, b="Doe"):
    print("Hello", a, "and", b)
# 1
hello("John", "Doe")
# 2
hello("Doe", "John")
# 3
hello(b="Doe", a="John")
# 4
hello("John")
# 5
hello(b="John")
```

`main()` for the flow, functions for the details

```
# main: WHAT the program does, step by step
def main():
    username = input("What's your name? ")   # 1. ask for a name
    hello()                                  # 2. say hello

# hello: HOW to say hello (write it later)
def hello():
    ...

main()   # call main to start the program
```

🤔 Scope: can `hello()` see `main()`'s variables?

```
def main():
    username = input("What's your name? ")
    hello()

def hello():
    print("Hello,", username)

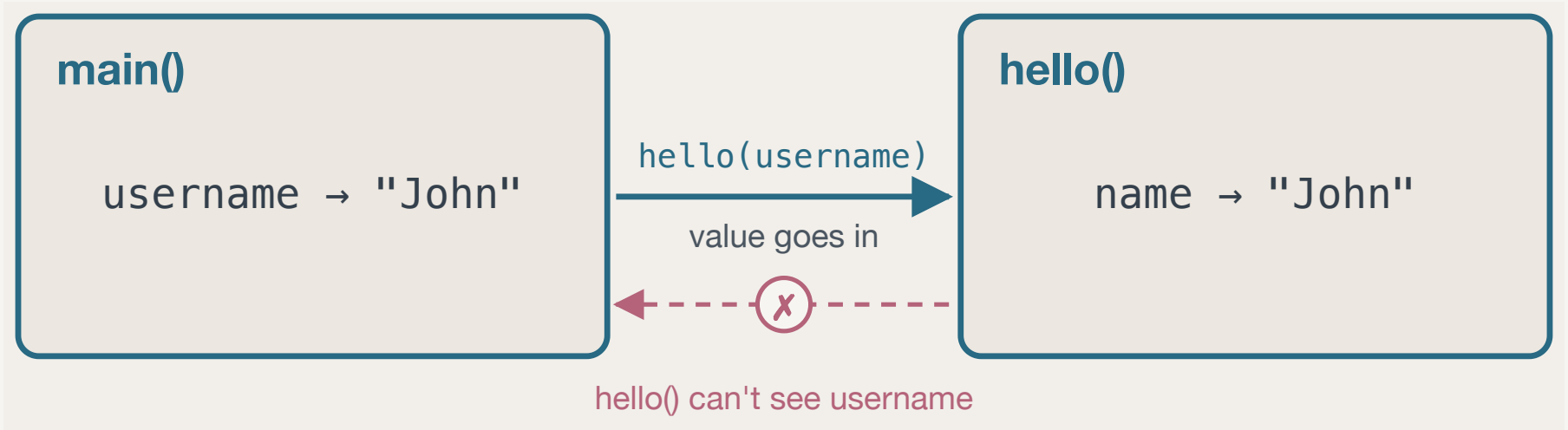
main()
```

Fixing scope by passing arguments

```
def main():
    username = input("What's your name? ")
    hello(username)   # name = username

def hello(name):      # parameter
    print("Hello,", name)

main()
```



`return` to hand a value back to the caller

```
def main():
    message = hello_message("John")  # message = "Hello, John"
    print(message)

def hello_message(name):
    msg = "Hello, " + name
    return msg                       # return the value of msg to the caller

main()
```

🤔 What gets printed?

```
def a():
    print("Hi")

def b():
    return "Hi"

x = a()
y = b()
print(x, y)
```

💻 6. Self-introduction 2

Requirements:

- Define `ask_name()`, which asks for the user's name with `input()` and returns it.
- Define `ask_age()`, which asks for the user's age and returns it.
- Define `introduce_message()`, which takes `name` and `age` as parameters and returns the string `Hello, my name is [name]. I am [age] years old.`
- `main()` is already written: it calls your functions in order and prints the message.

Expected Outputs:

```
What's your name? Emily
What's your age? 25
Hello, my name is Emily. I am 25 years old.
```

7. Calculator

```
Enter a number: 5
Enter another number: 3
8
```

🤔 Adding two inputs

```
def calculator():
    x = input("Enter a number: ")
    y = input("Enter another number: ")
    z = x + y

    print(z)

calculator()
```

`int()` to convert string to integer

```
def calculator():
    x = input("Enter a number: ")
    y = input("Enter another number: ")
    z = int(x) + int(y)   # convert strings to integers before adding

    print(z)

calculator()
```

Nested function calls: which style?

```
def calculator():
    x = input("Enter a number: ")
    y = input("Enter another number: ")
    z = int(x) + int(y)
    print(z)
```

VS.

```
def calculator():
    x = int(input("Enter a number: "))          # nest input inside int
    y = int(input("Enter another number: "))    # nest input inside int
    print(x+y)
```

VS.

```
def calculator():   # nest all the way
    print(int(input("Enter a number: ")) + int(input("Enter another number: ")))
```

`type()` to check a value's type

```
x = input("Enter a number: ")
type_x = type(x)
print(type_x)

y = int(x)
type_y = type(y)
print(type_y)
```

`int()`, `float()`, `str()` to convert between types

```
int("25")          # 25       string → integer
float("2.5")       # 2.5      string → decimal number
str(25)            # "25"     number → string

int("2.5")         # ValueError: use float() for decimals
```

`round()` to limit decimal places

```
round(2 / 3, 2)     # 0.67   2 decimal places
round(2.9)          # 3      no second argument: nearest whole number
int(2.9)            # 2      int() cuts off; it doesn't round
```

💻 8. Self-introduction 3

Requirements:

- Define `ask_birthyear()`, which asks for the user's birth year and returns it.
- Define `calc_age()`, which takes the birth year as an integer and returns the age in 2026.
- `ask_name()` and `introduce_message()` from Exercise 6 are already provided.
- Complete `main()`: fill in the blanks to convert the birth year to an integer and the age back to a string.

Expected Outputs:

```
What's your name? Emily
What's your birth year? 1998
Hello, my name is Emily. I am 28 years old.
```

Wrap up

- Variable: a name tag for a value; `=` assigns
- Function: a named block of code; arguments in, side effect and/or return value out
- Method: a function that belongs to a value, e.g., `answer.strip()`
- `def`: define a function; parameters receive the arguments of a call
- Scope: names stay inside their function; pass values in as arguments
- `return`: hand a value back to the caller
- `int()`, `float()`, `str()`: convert between types